ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN ĐA NGÀNH HƯỚNG CNPM

BÁO CÁO

IOT-Based Smart Irrigation in Greenhouse System

GVHD: Ths Nguyễn Phương Duy

SV thực hiện:	Nguyễn Miên Phú	2312658
	Đặng Vũ Duy	2310453
	Trần Doãn Hoàng Lâm	2311828
	Nguyễn Minh Tuấn	2313749
	Trần Tiến Khải	2311566

TP Hồ Chí Minh, Tháng 5 Năm 2026

Mục lục

1 Giới thiệu **6**
1.1 Mục tiêu . . . 6
1.2 Phạm vi . . . 6

2 Yêu cầu hệ thống **8**
2.1 Yêu cầu chức năng . . . 8
2.1.1 Nhóm chức năng quản lý, giám sát và tiền xử lý dữ liệu . . . 8
2.1.2 Nhóm chức năng điều khiển và vận hành . . . 8
2.1.3 Nhóm chức năng an toàn và cảnh báo . . . 8
2.1.4 Nhóm chức năng thông minh và tối ưu hóa . . . 8
2.2 Use case . . . 9
2.2.1 Use case 1: Đăng ký tài khoản . . . 9
2.2.2 Use case 2: Đăng nhập . . . 10
2.2.3 Use case 3: Quản lý nhà kính . . . 12
2.2.4 Use case 4: Thiết lập phân khu . . . 14
2.2.5 Use case 5: Thiết lập hàng cây . . . 14
2.2.6 Use case 6: Đăng ký thiết bị biên . . . 14
2.2.7 Use case 7: Cài đặt chế độ vận hành . . . 14
2.2.8 Use case 8: Điều khiển tưới thủ công . . . 16
2.2.9 Use case 9: Điều khiển tưới lập lịch . . . 18
2.2.10 Use case 10: Tự động điều tiết . . . 21
2.2.11 Use case 11: Giám sát trạng thái phân cấp . . . 23
2.2.12 Use case 12: Phát cảnh báo sự cố . . . 26
2.2.13 Use case 13: Dự báo lịch tưới tối ưu . . . 26
2.3 Yêu cầu phi chức năng . . . 26

3 Kiến trúc hệ thống **27**
3.1 Deployment Diagram . . . 28
3.1.1 Node Edge Device . . . 28
3.1.2 Node Fog Computer (Gateway) . . . 29
3.1.3 Node Cloud Server . . . 29
3.1.4 Node Client Devices (Personal Computer) . . . 29
3.2 Luồng dữ liệu . . . 30
3.2.1 Luồng dữ liệu thu thập (upstream) . . . 30
3.2.2 Luồng lệnh điều khiển (downstream) . . . 31
3.3 ERD . . . 32
3.3.1 Nhóm quản lý tài khoản và không gian . . . 33
3.3.2 Nhóm quản lý thiết bị và đo lường . . . 33
3.3.3 Nhóm điều khiển và tự động hóa . . . 33
3.3.4 Nhóm giám sát an toàn . . . 33
3.4 Cấu trúc MQTT Topic Tree . . . 34
3.4.1 Chi tiết phân luồng các nhóm chủ đề . . . 34
3.5 Sequence diagram . . . 35
3.6 Activity diagram . . . 35

4 Đánh giá và kiểm thử hệ thống **36**
4.1 Kiểm thử hệ thống 36
4.1.1 Kiểm thử phần mềm theo các phương pháp thiết kế Test Case 36
4.1.2 Đánh giá Hiệu năng Hệ thống (System Performance) 37
4.2 Đánh giá độ hiệu quả cây trồng 37
4.2.1 Phương pháp "Cái cân" 37
4.2.2 Phân tích đồ thị "Răng cưa" (Soil Moisture Profile) 37
4.2.3 Thiết lập mô hình đối chứng (A/B Testing Thực địa) 38
4.2.4 Đánh giá bằng Dữ liệu giả lập (Backtesting cho AI) 38
4.3 Kết quả 38

TÀI LIỆU THAM KHẢO **39**

Danh mục hình ảnh

1 Sơ đồ khối hệ thống 27
2 Sơ đồ triển khai hệ thống 28
3 Sơ đồ luồng dữ liệu của hệ thống 30
4 Sơ đồ luồng dữ liệu upstream của hệ thống 30
5 Sơ đồ luồng dữ liệu downstream của hệ thống 31
6 Sơ đồ ERD của cơ sở dữ liệu 32
7 Các topic của mqtt broker của hệ thống 34

Danh sách phân công

STT	MSSV	HỌ	TÊN	NHIỆM VỤ
1	2312658	Nguyễn Miên	Phú	•
2	2310453	Đặng Vũ	Duy	•
3	2311828	Trần Doãn Hoàng	Lâm	•
4	2313749	Nguyễn Minh	Tuấn	•
5	2311566	Trần Tiến	Khải	•

Lịch sử phiên bản

Ngày	Phiên bản	Thông tin	Author
12/03/2026	1.0	• Tạo template cho report • Viết phần 1 Giới thiệu	Nguyễn Miên Phú
14/03/2026	1.1	• Viết phần 2.1 Yêu cầu chức năng hệ thống • Viết UC Scenerio cho UC 1, 2, 3, 7, 8, 9, 10, 11	Nguyễn Miên Phú
20/03/2026	1.2	• Viết phần 3 Kiến trúc hệ thống (3.1, 3.2, 3.3, 3.4) • Viết phần 4 Đánh giá và kiểm thử hệ thống	Nguyễn Miên Phú
22/03/2026	1.3	• Chỉnh sửa phần 3.2, 3.3, 3.4 thống nhất cách gọi Fog Node • Chỉnh sửa UC 7, 8, 9, 10 thống nhất cách gọi Fog Node	Nguyễn Miên Phú

1 Giới thiệu

Theo FAO, dân số thế giới dự kiến sẽ đạt mức gần 10 tỷ người vào năm 2050, kéo theo yêu cầu sản lượng lương thực phải tăng thêm khoảng 56% để đáp ứng nhu cầu toàn cầu[2]. Thách thức này càng trở nên lớn hơn khi ngành nông nghiệp hiện đang tiêu thụ tới khoảng 70% dự trữ nước ngọt của thế giới[3].
Cùng với những tác động tiêu cực của biến đổi khí hậu, nguồn tài nguyên nước ngày càng cạn kiệt trong khi điều kiện thời tiết trở nên khắc nghiệt và khó dự đoán. Trong bối cảnh đó, hệ thống nhà kính đóng vai trò là một giải pháp bảo vệ cây trồng khỏi các yếu tố ngoại cảnh, giúp kiểm soát môi trường trồng trọt để tối ưu hóa năng suất trên diện tích đất hạn chế.
Dù vậy, các nhà kính truyền thống hiện nay đang đối mặt với những vấn đề lớn do việc tưới tiêu thường chỉ dựa trên kinh nghiệm thủ công hoặc lịch hẹn giờ cố định. Các phương pháp này gây ra hai hệ lụy chính:

- **Tưới quá mức:** Làm lãng phí nước, rửa trôi chất dinh dưỡng, tốn điện năng và dễ làm cây mắc các bệnh về nấm.
- **Tưới thiếu nước:** Làm cây thiếu nước, giảm năng suất và chất lượng nông sản.

Các hệ thống cũ này thiếu khả năng giám sát theo thời gian thực nên hoàn toàn thụ động trước những thay đổi bất ngờ. Để giải quyết vấn đề trên, nhóm nghiên cứu đề xuất hệ thống **Tưới tiêu thông minh dựa trên IoT (IoT-based Smart Irrigation)**.
Hệ thống này sẽ chuyển đổi phương pháp tưới từ việc đoán mò sang mô hình chính xác dựa trên dữ liệu (*data-driven*). Mục tiêu cốt lõi của đề tài là hướng tới tiêu chí cung cấp chính xác lượng nước cây cần, đồng thời giảm thiểu tối đa sự can thiệp thủ công của con người, cải thiện đáng kể hệ số cây trồng (*crop coefficient*) góp phần vào việc phát triển nền nông nghiệp bền vững.

1.1 Mục tiêu

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và triển khai một hệ thống quản lý tưới tiêu thông minh dựa trên nền tảng Internet vạn vật (IoT) dành cho nhà kính. Hệ thống hướng tới việc tự động hóa hoàn toàn quy trình tưới tiêu, tối ưu hóa hiệu suất sử dụng nước và ứng dụng các thuật toán dự báo để đảm bảo môi trường sinh trưởng lý tưởng nhất cho cây trồng, đồng thời giảm thiểu tối đa sai sót do yếu tố con người.

1.2 Phạm vi

Về phạm vi ứng dụng và không gian, hệ thống phục vụ các chủ nhà kính với mô hình quản lý phân cấp: Nhà kính, khu vực trồng và hàng cây. Đề tài giới hạn ở việc xây dựng một mô hình thu nhỏ để kiểm chứng giải pháp thay vì triển khai thực tế trên diện rộng. Quá trình thực nghiệm được tiến hành trên một số loại cây cụ thể như húng quế, bạc hà để đánh giá chính xác thuật toán tưới tiêu.
Về phạm vi chức năng, hệ thống tập trung vào bốn nhóm: giám sát, điều khiển, cảnh báo và tối ưu hóa. Thiết bị liên tục đo độ ẩm đất, nhiệt độ và độ ẩm không khí để tự động đóng/mở van và bơm theo ngưỡng, đồng thời hỗ trợ điều khiển thủ công qua Web. Hệ thống cũng tự động ngắt thiết bị và cảnh báo khi cạn nguồn nước, kết hợp ghi nhận dữ liệu lịch sử để đề xuất lịch tưới phù hợp với thói quen người dùng.

Về phạm vi công nghệ, hệ thống bám sát kiến trúc IoT tiêu chuẩn. Lớp thiết bị ngoại vi sử dụng vi điều khiển để thu thập dữ liệu và điều khiển thiết bị chấp hành. Việc truyền tải dữ liệu thời gian thực được thực hiện thông qua kết nối Wi-Fi và giao thức MQTT. Giao diện người dùng được xây dựng dưới dạng Web Application, với dữ liệu được quản lý tập trung trên nền tảng đám mây.
Về giới hạn của đề tài, hệ thống chỉ tập trung vào bài toán tưới tiêu ; các yếu tố như chiếu sáng, thông gió hay pha phân bón tự độngkhông thuộc phạm vi nghiên cứu. Mô hình giả định luôn có nguồn điện 220V và Internet ổn định, hỗ trợ khả năng hoạt động ngoại tuyến.

2 Yêu cầu hệ thống

2.1 Yêu cầu chức năng

2.1.1 Nhóm chức năng quản lý, giám sát và tiền xử lý dữ liệu

FR 1.1 Hệ thống vi điều khiển (Edge) phải thu thập dữ liệu theo chu kỳ thời gian thực (5s/lần) từ mạng lưới cảm biến được phân cấp: Nhiệt độ/độ ẩm không khí tại mỗi khu vực, độ ẩm đất tại mỗi hàng, và cảm biến mực nước tại bồn chứa tổng.

FR 1.2 Hệ thống phải có khả năng chuẩn hóa dữ liệu thô, đóng gói thành định dạng chuẩn (SenML) và Data Validation (Kalman filter) tại Fog Node trước khi truyền lên Server.

FR 1.3 Hệ thống phải hiển thị các thông số nhiệt độ, độ ẩm, độ ẩm đất, trạng thái van/máy bơm theo thời gian thực trên Dashboard (Web) theo cấu trúc quản lý phân cấp (Greenhouse $\rightarrow$ Zone $\rightarrow$ Row).

FR 1.4 Hệ thống phải lưu trữ dữ liệu cảm biến và nhật ký điều khiển vào cơ sở dữ liệu trên Cloud và file tại Fog Node để phục vụ việc tra cứu và huấn luyện mô hình học máy.

2.1.2 Nhóm chức năng điều khiển và vận hành

FR 2.1 Hệ thống phải cung cấp 4 chế độ vận hành độc lập (MANUAL, SCHEDULED, AUTO, AI-MODE). Việc chuyển đổi sang chế độ MANUAL sẽ tự động ghi đè và hủy bỏ tiến trình của các chế độ tự động khác.

FR 2.2 Hệ thống tự động bật van/bơm dựa trên ngưỡng cài đặt của từng hàng cây, bắt buộc áp dụng khoảng bù (ví dụ: bật khi $< 50\%$ và tắt khi $> 80\%$) để chống hiện tượng bật/tắt liên tục gây hỏng rơ-le.

FR 2.3 Hệ thống cho phép người dùng điều khiển bật/tắt, hẹn giờ tưới, thiết lập ngưỡng (profile) theo từng row hoặc bật chế độ AI từ xa qua web, có quyền ghi đè các quyết định của hệ thống tại greenhouse.

FR 2.4 Hệ thống phải cho phép người dùng tùy chỉnh độc lập các ngưỡng độ ẩm và lịch tưới cho từng hàng cây riêng biệt.

2.1.3 Nhóm chức năng an toàn và cảnh báo

FR 3.1 Hệ thống phải liên tục giám sát trạng thái cảm biến mực nước (30s/lần). Khi phát hiện cạn nước, phải ngay lập tức ($<$500ms) ngắt máy bơm để chống cháy nổ.

FR 3.2 Khi xảy ra sự cố cạn nước, hệ thống tự động khóa toàn bộ các lệnh bơm. Hệ thống chỉ cho phép kích hoạt lại khi có lệnh xác nhận an toàn từ người dùng.

FR 3.3 Hệ thống phải gửi cảnh báo theo thời gian thực (*Push Notification/SMS*) tới người dùng khi xảy ra sự cố: cạn nước bồn chứa, mất kết nối thiết bị, hoặc nhiệt độ khu vực vượt quá giới hạn an toàn ($> 40^{\circ}$C).

2.1.4 Nhóm chức năng thông minh và tối ưu hóa

FR 4.1 Hệ thống phải tích hợp mô hình học máy để phân tích dữ liệu lịch sử nhiệt độ và độ ẩm, từ đó dự báo tốc độ suy giảm độ ẩm đất và tự động tính toán thời gian mở van tưới tối ưu, tránh ngập úng.

FR 4.2 Hệ thống phải ghi nhận tần suất, khung giờ và thời lượng người dùng can thiệp tưới thủ công trong vòng 7 ngày gần nhất.

FR 4.3 Dựa trên dữ liệu tổng hợp từ FR 4.2 và FR 4.1, hệ thống tự động đưa ra các đề xuất yêu cầu người dùng tinh chỉnh lại ngưỡng tưới hoặc lịch tưới cho phù hợp với chu kỳ thực tế của cây trồng.

2.2 Use case

2.2.1 Use case 1: Đăng ký tài khoản

USE CASE 1: Đăng ký tài khoản

Usecase name	Đăng ký tài khoản
Description	Use case này mô tả quá trình đăng ký một định danh mới trên hệ thống
Actors	Chủ nhà kính
Preconditions	• Thiết bị của người dùng có kết nối Internet. • Người dùng chưa đăng nhập vào hệ thống. • Email người dùng nhập vào chưa từng tồn tại trên cơ sở dữ liệu của hệ thống.
Trigger	Người dùng nhấn vào nút đăng ký tài khoản
Post-conditions	**Success:** Bản ghi mới của người dùng được lưu an toàn trong bảng USERS với mật khẩu đã được mã hóa (password_hash). **Failure:** Người dùng được thông báo về mã lỗi cụ thể.

Usecase name	Đăng ký tài khoản (Tiếp theo)
Main flow	1. Người dùng truy cập vào trang chủ hoặc trang đăng nhập và nhấn chọn nút "Đăng ký". 2. Hệ thống hiển thị form đăng ký yêu cầu nhập: Họ tên, email, và mật khẩu. 3. Người dùng nhập đầy đủ thông tin và nhấn nút "Xác nhận đăng ký". 4. Hệ thống kiểm tra tính hợp lệ dữ liệu (Format email, mật khẩu $>$ 8 ký tự). 5. Hệ thống truy vấn CSDL để kiểm tra Email đã tồn tại hay chưa. 6. Hệ thống thực hiện mã hóa mật khẩu thành chuỗi *password_hash*. 7. Hệ thống tạo một bản ghi mới trong bảng users với role mặc định là OWNER. 8. Hệ thống chuyển hướng sang Dashboard và thông báo: "Đăng ký thành công".
Alternative flow	
Exceptions	• E1. Lỗi định dạng dữ liệu (Bước 4): Hệ thống chặn gửi request, bôi đỏ các ô lỗi và yêu cầu nhập lại. • E2. Email đã tồn tại (Bước 5): Nếu hệ thống phát hiện email đã được đăng ký trong bảng USERS, hệ thống ngừng xử lý và hiển thị thông báo: "Email này đã được sử dụng. Vui lòng đăng nhập hoặc sử dụng email khác". Luồng quay về Bước 2.

2.2.2 Use case 2: Đăng nhập

USE CASE 2: Đăng nhập

Usecase name	Đăng nhập
Description	Use case này mô tả quá trình đăng nhập vào một tài khoản trên hệ thống
Actors	Chủ nhà kính và quản trị viên hệ thống.
Preconditions	• Tài khoản của người dùng đã được tạo trong cơ sở dữ liệu. • Người dùng đang ở trạng thái chưa đăng nhập (hoặc Token cũ đã hết hạn).

Usecase name	Đăng nhập (Tiếp theo)
Trigger	Người dùng nhấn vào nút đăng nhập trên trang chủ
Post-conditions	**Success:** Phiên làm việc (Session) được thiết lập thành công tại phía Client (Browser/App lưu trữ JWT). **Failure:** Người dùng được thông báo về mã lỗi cụ thể.
Main flow	1. Người dùng truy cập vào trang Đăng nhập (Login Page) của ứng dụng. 2. Hệ thống hiển thị biểu mẫu yêu cầu nhập Email và Mật khẩu. 3. Người dùng nhập thông tin và nhấn nút "Đăng nhập". 4. Hệ thống (Backend) tiếp nhận yêu cầu và kiểm tra định dạng dữ liệu đầu vào. 5. Hệ thống truy vấn CSDL (bảng USERS) để tìm bản ghi có Email khớp với dữ liệu người dùng cung cấp. 6. Hệ thống sử dụng thuật toán băm (Hash) để xử lý mật khẩu vừa nhập và đối chiếu với trường password_hash lưu trong CSDL. 7. Kết quả: Mật khẩu hoàn toàn trùng khớp. 8. Hệ thống khởi tạo một JSON Web Token (JWT) chứa các thông tin định danh (Payload) bao gồm: user_id, role (ADMIN hoặc OWNER), và thời gian hết hạn (Expiration Time). 9. Hệ thống trả JWT về cho Client và tự động chuyển hướng (Redirect) người dùng về trang Dashboard
Alternative flow	• A1. Tại bước 2 chuyển thẳng đến Dashboard nếu session còn hiệu lực.

Usecase name	Đăng nhập (Tiếp theo)
Exceptions	• E1. Ngăn chặn Brute-force (Luồng bảo mật): Nếu người dùng nhập sai mật khẩu quá 5 lần liên tiếp trong vòng 5 phút, hệ thống tự động khóa tạm thời IP hoặc tài khoản đó trong 15 phút để chống tấn công dò mật khẩu. • E2. Email đã tồn tại (Bước 5): Nếu hệ thống phát hiện email đã được đăng ký trong bảng USERS, hệ thống ngừng xử lý và hiển thị thông báo: "Email này đã được sử dụng. Vui lòng đăng nhập hoặc sử dụng email khác". Luồng quay về Bước 2. • E3. Nếu Email không tồn tại hoặc Mật khẩu sau khi băm không khớp, hệ thống từ chối truy cập và hiển thị thông báo chung: "Email hoặc mật khẩu không chính xác". (Hệ thống quay về Bước 2).

2.2.3 Use case 3: Quản lý nhà kính

USE CASE 3: Quản lý nhà kính

Usecase name	Quản lý nhà kính
Description	Use case này mô tả quá trình Chủ nhà kính xem danh sách, khởi tạo mới, chỉnh sửa thông tin hoặc xóa bỏ một nhà kính. Hệ thống sẽ tự động đồng bộ cấu trúc lưu trữ và không gian giao tiếp dữ liệu tương ứng.
Actors	Chủ nhà kính (Owner).
Preconditions	• Chủ nhà kính đã đăng nhập thành công vào hệ thống và có Token (JWT) hợp lệ. • Giao diện đang ở trang bảng điều khiển chính (Main Dashboard).
Trigger	Người dùng nhấn vào nút Quản lý Nhà kính trên menu điều hướng.
Post-conditions	**Success:** Thông tin nhà kính được cập nhật chính xác vào CSDL, đảm bảo tính toàn vẹn dữ liệu. Không gian giao tiếp MQTT cho nhà kính đó được cấp phát hoặc thu hồi đồng bộ với CSDL. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.

Usecase name	Quản lý nhà kính (Tiếp theo)
Main flow	1. Chủ nhà kính truy cập vào mục "Quản lý Nhà kính" trên menu điều hướng. 2. Hệ thống truy vấn CSDL và hiển thị danh sách các Nhà kính hiện có trực thuộc tài khoản của người dùng (dựa trên user_id trong Token). 3. Chủ nhà kính nhấn nút "Thêm nhà kính mới". 4. Hệ thống (Backend) tiếp nhận yêu cầu và kiểm tra định dạng dữ liệu đầu vào. 5. Hệ thống hiển thị biểu mẫu (Form) yêu cầu nhập các thông tin: Tên nhà kính, vị trí. 6. Chủ nhà kính nhập thông tin và nhấn "Lưu lại". 7. Hệ thống (Backend) tiếp nhận yêu cầu và kiểm tra tính hợp lệ của dữ liệu (Tên không được để trống, độ dài hợp lệ). 8. Hệ thống tự động sinh ra một mã định danh greenhouse_id cùng với user_id trong token thành một khóa chính. 9. Hệ thống tự động định nghĩa một cấu trúc MQTT topic dành riêng cho nhà kính này và cập nhật quyền (ACL - Access Control List) trên MQTT Broker để chuẩn bị đón luồng dữ liệu. 10. Hệ thống lưu bản ghi mới vào bảng greenhouses trong CSDL. 11. Hệ thống đóng biểu mẫu, tải lại danh sách nhà kính và hiển thị thông báo: "Khởi tạo nhà kính thành công".
Alternative flow	• A1. Cập nhật thông tin: Tại bước 3, người dùng chọn một nhà kính có sẵn và nhấn "Chỉnh sửa". Hệ thống nạp dữ liệu cũ lên biểu mẫu. Người dùng sửa đổi và nhấn "Cập nhật". Hệ thống lưu thay đổi vào CSDL và thông báo thành công. • A2. Xóa nhà kính: Tại bước 3, người dùng chọn một nhà kính và nhấn "Xóa". Hệ thống hiển thị cảnh báo đỏ: "Hành động này sẽ xóa toàn bộ Zone, Row, thiết bị và Lịch sử dữ liệu thuộc nhà kính này. Bạn có chắc chắn không?" yêu cầu nhập lại mật khẩu để xác nhận. Sau khi xác nhận, hệ thống thực hiện Cascading Delete trong CSDL để dọn dẹp toàn bộ dữ liệu liên quan, đồng thời thu hồi quyền truy cập MQTT Topic của nhà kính đó trên Broker.

Usecase name	Quản lý nhà kính (Tiếp theo)
Exceptions	• E1. Lỗi xác thực dữ liệu (Bước 6): Nếu người dùng bỏ trống tên nhà kính, hệ thống từ chối lưu, đánh dấu đỏ ô nhập liệu và yêu cầu điền đầy đủ thông tin.

2.2.4 Use case 4: Thiết lập phân khu

2.2.5 Use case 5: Thiết lập hàng cây

2.2.6 Use case 6: Đăng ký thiết bị biên

2.2.7 Use case 7: Cài đặt chế độ vận hành

USE CASE 7: Cài đặt chế độ vận hành

Usecase name	Cài đặt chế độ vận hành
Description	Use case này mô tả quá trình chủ nhà kính thiết lập phương thức hoạt động (Auto, AI, Manual, Scheduled) và đồng bộ cấu hình xuống Fog Node để lưu trữ.
Actors	Chủ nhà kính, Cloud Server, Fog Node, Edge Node (Yolo:bit).
Preconditions	• Người dùng đã đăng nhập và đang ở trang Dashboard. • Thiết bị Edge quản lý hàng cây tương ứng đã được đăng ký trên hệ thống. • Kết nối mạng nội bộ giữa Edge Node và Fog Node đang hoạt động bình thường. • Kết nối mạng giữa Fog Node và Cloud Server đang hoạt động bình thường.
Trigger	Người dùng nhấp chọn một hàng cây trên Dashboard và mở bảng cấu hình vận hành.
Post-conditions	**Success:** Trạng thái current_mode của hàng cây trên CSDL và trong bộ nhớ của Fog Node hoàn toàn trùng khớp. Hệ thống vận hành theo đúng cơ chế đã được lựa chọn. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.

Usecase name	Cài đặt chế độ vận hành (Tiếp theo)
Main flow	1. Người dùng nhấp chọn một hàng cây trên Dashboard và mở bảng cấu hình vận hành. 2. Người dùng chọn chế độ (ở luồng này mô tả chế độ AUTO). 3. Hệ thống hiển thị hai trường dữ liệu yêu cầu nhập: threshold_min (Ngưỡng bật bơm). threshold_max (Ngưỡng tắt bơm). 4. Người dùng nhập số liệu và nhấn nút "Áp dụng cấu hình". 5. Frontend kiểm tra logic đầu vào (Đảm bảo min $<$ max và nằm trong dải 0-100). Nếu hợp lệ, gửi API Request lên Backend. 6. Backend (Server) tiếp nhận, cập nhật các trường current_mode, threshold_min, threshold_max vào bảng rows trong Cơ sở dữ liệu. 7. Backend đóng gói cấu hình thành một bản tin JSON chuẩn SenML và publish vào đúng Shared Topic của hàng đó qua giao thức MQTT với QoS = 1 8. Fog Node nhận được bản tin cấu hình. 9. Fog Node lập tức ghi cấu hình mới này vào bộ nhớ. 10. Fog Node thay đổi nhánh logic theo biến trạng thái AUTO (trong Node-RED). 11. Fog Node publish một bản tin ACK ngược lại topic trạng thái trên MQTT Broker. 12. Backend nhận được bản tin ACK, thông qua WebSockets đẩy một sự kiện (Event) xuống giao diện Frontend của người dùng. 13. Dashboard hiển thị thông báo popup: "Đã đồng bộ cấu hình AUTO xuống thiết bị thành công" và cập nhật biểu tượng trạng thái của hàng cây sang màu xanh và hiện thị chế độ hiện tại Auto.

Usecase name	Cài đặt chế độ vận hành (Tiếp theo)
Alternative flow	• A1. Chuyển sang chế độ THÔNG MINH (AI Mode): Tại Bước 2, người dùng chọn AI Mode. Frontend ẩn các ô nhập ngưỡng Min/Max. Khi nhấn "Áp dụng", Backend cập nhật CSDL và publish bản tin vào Shared Topic. Fog Node nhận tin, lưu cấu hình, và chuyển sang nhánh logic dành cho chế độ AI. Fog Node chờ nhận lệnh bật/tắt định thời từ AI trên Cloud Server gửi về. Hệ thống phân tích (Cloud AI) đọc data của hàng cây này để tính toán (Chuyển tiếp sang UC13: Dự báo lịch tưới tối ưu). • A2. Chuyển sang chế độ THỦ CÔNG (MANUAL Mode): Tại Bước 2, người dùng chọn MANUAL. Backend cập nhật CSDL và publish bản tin vào Shared Topic. Fog Node chuyển sang nhánh logic dành cho chế độ MANUAL đưa máy bơm về trạng thái chờ và chỉ kích hoạt Relay khi có lệnh điều khiển trực tiếp từ người dùng (Chuyển tiếp sang UC08: Điều khiển tưới thủ công). • A3. Chuyển sang chế độ LẬP LỊCH (SCHEDULED mode) Tại Bước 2, người dụng chọn SCHEDULED. Hệ thống hiển thị hai trường dữ liệu yêu cầu nhập: start_time (thời gian bắt đầu tưới), duration (thời lượng tưới). (Chuyển tiếp sang UC09: Điều khiển tưới lập lịch).
Exceptions	• E1. Lỗi logic đầu vào (Bước 5): Nếu người dùng nhập threshold_min = 80 và threshold_max = 50, Frontend chặn request, bôi đỏ ô nhập liệu và hiển thị cảnh báo: "Ngưỡng bật bơm phải nhỏ hơn ngưỡng tắt bơm". • E2. Fog Node mất kết nối với Cloud Server: Tại Bước 7, Backend publish lệnh MQTT nhưng đợi quá thời gian Timeout (ví dụ 5 giây) mà không nhận được bản tin ACK từ Fog Node (Bước 10). Backend vẫn giữ lại cấu hình trong CSDL nhưng gắn cờ trạng thái là Pending Sync (Chờ đồng bộ). Dashboard hiển thị cảnh báo màu vàng: "Cấu hình đã lưu trên Cloud nhưng thiết bị hiện đang mất kết nối. Cấu hình sẽ tự động đồng bộ khi thiết bị online trở lại." Khi Fog Node kết nối trở lại, quá trình re-connect sẽ tự động kéo bản cấu hình này về để cập nhật (nhờ tính năng Retain của MQTT Broker).

2.2.8 Use case 8: Điều khiển tưới thủ công

USE CASE 8: Điều khiển tưới thủ công

Usecase name	Điều khiển tưới thủ công
Description	Use case này mô tả quá trình chủ nhà kính can thiệp bật/tắt máy bơm trực tiếp theo thời gian thực, đi kèm các cơ chế tự động bảo vệ phần cứng (Timeout) và xử lý xung đột (Override) với các chế độ tự động.
Actors	Chủ nhà kính, Cloud Server, Fog Node, Edge Node (Yolo:bit).
Preconditions	• Người dùng đã đăng nhập và Fog Node đang online. • Relay máy bơm không bị khóa cứng (Hard-locked) bởi sự cố cạn bể chứa từ UC10.
Trigger	Người dùng nhấp chọn một hàng cây trên Dashboard và chọn bật/tắt bơm.
Post-conditions	**Success:** Trạng thái bật/tắt của máy bơm thực tế khớp với hiển thị trên Dashboard. Mọi sự kiện can thiệp thủ công (bật, tắt, hoặc quá giờ) đều được ghi nhận vào bảng control_logs nhằm phục vụ việc tính toán lượng nước tiêu thụ và kiểm toán lại sau này. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.
Main flow	1. Người dùng chọn một hàng cây trên bảng điều khiển (Dashboard) và nhấn nút "BẬT BƠM" (Turn ON). 2. Frontend gửi request lên Server. Server kiểm tra các điều kiện an toàn cơ bản và publish một bản tin MQTT (QoS 1) mang lệnh PUMP_ON, timeout_mins: 30 vào Shared Topic. 3. Fog Node nhận tin, publish bản tin vào Internal Topic để vi mạch lập tức đóng Relay kích hoạt máy bơm. 4. Fog Node tự động khởi chạy một bộ đếm thời gian đếm ngược (Timeout Timer) độc lập (VD: 30 phút). 5. Fog Node publish một bản tin ACK ngược lại. 6. Server nhận ACK, đẩy dữ liệu qua WebSocket để Dashboard cập nhật hiệu ứng "Đang tưới" màu xanh lá. 7. Khi đã tưới đủ, người dùng nhấn nút "TẮT BƠM" (Turn OFF) trên Dashboard. 8. Server publish lệnh PUMP_OFF vào Shared Topic. 9. Fog Node nhận tin, publish bản tin vào Internal Topic để Yolo:bit ngắt Relay máy bơm và đồng thời hủy bộ đếm ở bước 4. 10. Fog Node publish bản tin ACK ngược lại.

Usecase name	Điều khiển tưới thủ công (Tiếp theo)
Alternative flow	• A1. Kích hoạt bảo vệ quên tắt: Tại Bước 7, người dùng đóng trình duyệt và QUÊN không bấm nút tắt. Bộ đếm Timeout Timer dưới mạch tại Fog Node (thiết lập ở Bước 4) chạy về 0. Fog Node gửi tin vào Internal Topic để Yolo:bit ngắt Relay máy bơm để chống cháy bơm và úng rễ cây. Fog Node publish bản tin Alert lên Cloud Server. Cloud Server lưu sự cố, đồng thời gửi một Cảnh báo (Alert) đến điện thoại/email người dùng: "Hệ thống đã tự động ngắt bơm tại Hàng 1 do vượt quá thời gian tưới an toàn (30 phút)."
Exceptions	• E1. Xung đột lệnh điều khiển: Hàng cây đang ở chế độ AI Mode. Người dùng đột ngột bấm nút "BẬT BƠM" (Manual). Server lập tức ngắt tiến trình (Thread/Job) của AI đang chạy ngầm. Server cập nhật current_mode của Row đó về MANUAL trong Database. Dashboard hiển thị thông báo popup: "Chế độ AI đã bị ngưng, hệ thống đã chuyển sang điều khiển thủ công theo lệnh của bạn." (Luồng tiếp tục tại Bước 2). • E2. Vi phạm chốt khóa phần cứng: Người dùng bấm "BẬT BƠM" nhưng trước đó Fog Node đã gửi cờ báo hiệu bể chứa cạn nước. Hành động: Backend từ chối gửi lệnh MQTT đi. Giao diện từ chối thao tác và nháy đỏ: "Lỗi: Không thể bật bơm. Bể chứa đang cạn nước, hệ thống đã khóa an toàn." (Bắt buộc người dùng phải châm nước và unlock trên Dashboard mới được dùng tiếp). • E3. Mất kết nối khi đang bơm: Ở Bước 5, bơm đã bật, nhưng đột ngột rớt mạng Internet. Người dùng bấm TẮT nhưng lệnh không gửi xuống được. Máy bơm vẫn sẽ tự động ngắt sau 30 phút.

2.2.9 Use case 9: Điều khiển tưới lập lịch

USE CASE 9: Điều khiển tưới lập lịch

Usecase name	Điều khiển tưới lập lịch
Description	Use case này mô tả quá trình chủ nhà kính thiết lập các mốc thời gian và thời lượng tưới cụ thể trong ngày cho từng hàng cây.
Actors	Chủ nhà kính, Cloud Server, Fog Node, Edge Node (Yolo:bit).
Preconditions	• Fog Node đang ở trạng thái Online và đã đồng bộ thời gian thực qua giao thức NTP (Network Time Protocol). • Chủ nhà kính đã đăng nhập và đang thao tác trên Dashboard. • Máy bơm không bị khóa cứng (Hard-locked) bởi sự cố cạn bể chứa từ UC10.
Trigger	Người dùng nhấp chọn một hàng cây trên Dashboard và mở bảng cấu hình vận hành.
Post-conditions	**Success:** Trạng thái bật/tắt của máy bơm thực tế khớp với hiển thị trên Dashboard. Mọi sự kiện (bật, tắt) đều được ghi nhận vào bảng CONTROL_LOGS nhằm phục vụ việc tính toán lượng nước tiêu thụ và audit lại sau này. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.

Usecase name	Điều khiển tưới lập lịch (Tiếp theo)
Main flow	1. Người dùng chọn một hàng cây, chọn chế độ SCHEDULED và nhấn "Thêm lịch tưới mới". 2. Hệ thống hiển thị biểu mẫu yêu cầu nhập: Giờ bắt đầu (start_time), Thời lượng tưới theo phút (duration_mins). 3. Người dùng điền thông tin và nhấn "Áp dụng cấu hình". 4. Frontend kiểm tra tính hợp lệ của dữ liệu và gửi API Request. 5. Backend tiếp nhận, truy vấn CSDL để đảm bảo lịch mới không bị trùng lặp với các lịch đang Active khác của hàng cây này này. 6. : Backend cập nhật CSDL current_mode = "SCHEDULED" trong bảng rows. Đồng thời thêm bản ghi mới vào bảng schedules. 7. Backend truy xuất toàn bộ danh sách các lịch đang is_active=true của hàng cây đó, đóng gói thành một mảng JSON và publish qua MQTT (QoS 1) vào Shared Topic cấu hình của Fog Node. 8. Fog Node nhận gói tin, lập tức lưu lịch này vào bộ nhớ. 9. Fog Node publish bản tin ACK ngược lại. 10. Backend nhận ACK, thông qua WebSockets đẩy sự kiện cập nhật giao diện: "Thiết lập lịch tưới thành công", danh sách lịch mới xuất hiện trên Dashboard.
Alternative flow	• A1. Bật/tắt một lịch tưới đã có: Tại Bước 3, người dùng gạt công tắc is_active từ ON sang OFF của một lịch cũ trên Dashboard. Backend cập nhật CSDL (is_active=false) và đóng gói lại mảng JSON chỉ chứa các lịch còn ACTIVE đẩy xuống Fog Node. Fog Node ghi đè các lịch cũ thành các lịch mới được ACTIVE vào bộ nhớ.

Usecase name	Điều khiển tưới lập lịch (Tiếp theo)
Exceptions	• E1. Xung đột thời gian: Tại Bước 5, Backend phát hiện thời gian người dùng cấu hình bị đè lên một lịch đã có (VD: Lịch A tưới 06:00-06:30, người dùng tạo Lịch B lúc 06:15). Backend từ chối thiết lập, Frontend hiển thị cảnh báo đỏ: "Lỗi: Lịch tưới bị trùng lặp thời gian hoạt động. Vui lòng chọn khung giờ khác." • E2. Xung đột với điều khiển thủ công (Tương tác với UC08): Tình huống: Đến 06:00, Fog Node đang ra lệnh (publish vào Internal Topic) cho Edge Node bật bơm theo SCHEDULED. Nhưng lúc 06:05, người dùng nhấn nút TẮT BƠM (Turn OFF) trên Dashboard. Fog Node nhận lệnh, lập tức ra lệnh cho Edge Node ngắt Relay máy bơm, hủy bỏ quá trình đếm ngược 15 phút của SCHEDULED. Backend chuyển current_mode của hàng cây đó sang MANUAL. • E3. Fog Node đang Offline (mất kết nối với Cloud Server): Tại Bước 7, Backend publish gói tin nhưng đợi hết Timeout không thấy ACK từ Fog Node. Backend lưu thành công vào CSDL nhưng gán cờ trạng thái là Pending Sync (Chờ đồng bộ). Dashboard hiển thị icon màu vàng cảnh báo. Khi Fog Node kết nối lại, quá trình Re-connect sẽ tự động kéo bản cấu hình mới nhất này về để cập nhật giờ tưới. • E4. Cạn bể chứa khi đến giờ hẹn (Tương tác với UC10): Tại Luồng thực thi ngầm Bước 2, đến giờ tưới nhưng cảm biến báo cạn nước. Fog Node hủy bỏ việc bật bơm, ghi log lỗi, và publish cảnh báo WATER_LOW_LOCKED lên Server để báo cho Chủ nhà kính biết lịch tưới đã bị bỏ qua do cạn nước.

2.2.10 Use case 10: Tự động điều tiết

USE CASE 10: Tự động điều tiết

Usecase name	Tự động điều tiết
Description	Use case này mô tả quá trình Fog Node có khả năng tự vận hành vòng lặp tưới tiêu (dựa trên cấu hình đã lưu), đồng thời cung cấp lớp bảo vệ phần cứng mức thấp nhất (Low-level Hardware Protection) nhằm vô hiệu hóa các lệnh nguy hiểm từ Cloud hoặc User.
Actors	Fog Node, Edge Node.

Usecase name	Tự động điều tiết (Tiếp theo)
Preconditions	• Fog Node và Edge Node đã được cấp nguồn và đang hoạt động. • Cấu hình ngưỡng (threshold_min, threshold_max) đã được lưu thành công vào bộ nhớ.
Trigger	Luồng này diễn ra ngầm liên tục khi current_mode của thiết bị đang là AUTO.
Post-conditions	Máy bơm tuyệt đối không bao giờ bị cháy do dry-run. Cây trồng không bị chết khát khi đứt cáp quang/rớt mạng. Trạng thái và các cảnh báo (Alerts) luôn được đồng bộ minh bạch lên Cloud ngay khi đường truyền khả dụng. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.
Main flow	1. Fog Node liên tục đọc dữ liệu từ cảm biến độ ẩm đất theo chu kỳ (ví dụ: 10 giây/lần) từ Edge Node thông qua Internal Topic 2. Fog Node đối chiếu giá trị cảm biến hiện tại với ngưỡng đã lưu trong bộ nhớ: Nếu Độ ẩm $<$ threshold_min: Fog Node xuất tín hiệu đóng Relay để BẬT máy bơm. Nếu Độ ẩm $>$ threshold_max: Fog Node xuất tín hiệu ngắt Relay để TẮT máy bơm qua Internal Topic. 3. Mỗi khi có sự thay đổi trạng thái Relay, Fog Node publish bản tin MQTT (QoS 1, kèm cờ Retain=true) lên Broker (ví dụ với nội dung: "status": "ON/OFF", "actor": "AUTO").
Alternative flow	

Usecase name	Tự động điều tiết (Tiếp theo)
Exceptions	• E1. Chốt khóa cạn nước. Cảm biến mức nước trong bể chứa báo cạn nước. Fog Node lập tức ra lệnh cho Edge Node ngắt Relay máy bơm (nếu đang chạy) và kích hoạt trạng thái KHÓA CỨNG (Hard-lock). Trạng thái Hard-lock từ chối hoàn toàn mọi lệnh BẬT bơm từ người dùng (UC08 - Manual), từ Lịch hẹn giờ (UC09 - SCHEDULED) và từ AI. Fog Node publish mã lỗi WATER_LOW_LOCKED. Backend lưu vào bảng ALERTS và bắn Push Notification/Email khẩn cấp cho User: "Bơm đã bị khóa an toàn do cạn bể chứa". Hệ thống chỉ mở khóa khi bể được châm đủ nước VÀ người dùng nhấn nút "Reset/Unlock" trên Dashboard. • E2. Rớt mạng. Fog Node mất kết nối với Cloud Server (Cloud MQTT Broker). Nếu thiết bị đang ở chế độ phụ thuộc Cloud (như AI Mode), Edge tự động hạ cấp về chế độ AUTO cục bộ. Nó sử dụng ngưỡng Min/Max cuối cùng còn lưu trong bộ nhớ để duy trì sự sống cho cây trồng. Khi có mạng trở lại, Fog Node tự động Re-connect, publish báo cáo và đồng bộ lại cấu hình mới nhất từ Backend (nếu người dùng có đổi cấu hình trong lúc nó rớt mạng). • E3. Bảo vệ chống tưới tràn do lỗi cảm biến. Fog Node đọc dữ liệu độ ẩm đất thông qua Internal Topic và nhận thấy giá trị phi vật lý. Fog Node vô hiệu hóa vòng lặp kích hoạt tự động của chính cảm biến đó để tránh bơm nước liên tục gây úng rễ cây. Fog Node publish mã lỗi SENSOR_MALFUNCTION. Backend lưu bảng ALERTS báo thợ sửa chữa. Trong lúc chờ sửa cảm biến, người dùng vẫn có thể dùng UC08 (Manual Mode) để tự bấm tưới tay nếu họ quan sát thấy đất khô.

2.2.11 Use case 11: Giám sát trạng thái phân cấp

USE CASE 11: Giám sát trạng thái phân cấp

Usecase name	Giám sát trạng thái phân cấp
Description	Use case này mô tả quá trình người dùng theo dõi các thông số môi trường và trạng thái thiết bị theo cấu trúc cây (Nhà kính → Phân khu → Hàng cây) dưới hai góc nhìn: Thời gian thực và biểu đồ lịch sử.

Usecase name	Giám sát trạng thái phân cấp (Tiếp theo)
Actors	Chủ nhà kính, Cloud Server, Fog Node, Edge Node.
Preconditions	• Người dùng đã xác thực thành công (có JWT Token hợp lệ). • Fog Node, Edge Node đang hoạt động và liên tục gửi luồng dữ liệu lên Cloud.
Trigger	Người dùng chọn xem lịch sử hoặc đang ở dashboard chính
Post-conditions	**Success:** Người dùng nắm bắt được chính xác 100% tình trạng vật lý hiện tại của nhà kính thông qua giao diện trực quan. Giao diện duy trì được tính tương tác cao và khả năng chịu lỗi mạng tốt. **Failure:** Người dùng được thông báo về mã lỗi cụ thể.

Usecase name	Giám sát trạng thái phân cấp (Tiếp theo)
Main flow	1. Cấp độ nhà kính: Người dùng truy cập vào trang bảng điều khiển . Backend kiểm tra Token và trả về danh sách các nhà kính mà người dùng sở hữu. 2. Người dùng nhấp chọn "Nhà kính A". Giao diện khởi tạo kết nối WebSocket với Backend để lắng nghe kênh dữ liệu của nhà kính A. 3. Cấp độ phân khu: Giao diện hiển thị danh sách các phân khu bên trong nhà kính A. Tại đây, hiển thị các thông số: Nhiệt độ và độ ẩm không khí (dữ liệu lấy từ các cảm biến dùng chung của Zone). 4. Cấp độ hàng cây: Người dùng nhấp mở rộng "Zone 1". Hệ thống hiển thị chi tiết các hàng cây thuộc "Zone 1". Tại đây, hiển thị các thông số vi mô: Độ ẩm đất cục bộ, trạng thái máy bơm (đang bật hay tắt), loại cây đang trồng và chế độ vận hành hiện tại (AUTO, MANUAL, SCHEDULED, AI). 5. Hot Path: Mỗi khi có dữ liệu mới (ví dụ: độ ẩm đất thay đổi từ 65% lên 70%), dữ liệu đi qua Cloud Broker → Redis Pub/Sub → WebSocket. Các con số trên màn hình của người dùng sẽ tự động nhảy mà không cần tải lại trang. 6. Truy xuất lịch sử - Cold Path: Người dùng nhấp vào biểu tượng "Biểu đồ" tại một hàng cây cụ thể. 7. Frontend gửi một REST API Request: Backend truy vấn CSDL MongoDB (Time-series Collection), tính toán và trả về mảng dữ liệu. 8. Dashboard vẽ biểu đồ đường thể hiện sự biến thiên của độ ẩm đất trong 24 giờ qua, kết hợp với các cột mốc thể hiện những lúc máy bơm được bật/tắt (lấy từ bảng CONTROL_LOGS).
Alternative flow	

Usecase name	Giám sát trạng thái phân cấp (Tiếp theo)
Exceptions	• E1. Thiết bị mất kết nối: Khi Backend phát hiện Fog Node đã ngắt kết nối MQTT (quá Timeout hoặc nhận bản tin LWT - Last Will and Testament). Backend đẩy sự kiện "Offline" qua WebSocket. Dashboard tự động làm xám (gray-out) thẻ thông tin của hàng cây/khu vực đó, hiển thị icon mất kết nối, và dòng chữ: "Cập nhật lần cuối lúc [Thời gian]" thay vì hiển thị trạng thái Real-time. • E2. Cảm biến báo lỗi hoặc cạn nước: Dữ liệu trả về có chứa mã lỗi (VD: Cạn nước bể chứa). Trạng thái hàng cây/khu vực lập tức chuyển sang viền đỏ. Icon máy bơm chuyển sang biểu tượng "Khóa" (Locked). Giao diện hiển thị banner cảnh báo khẩn cấp để thu hút sự chú ý của người dùng. • E3. Mất kết nối mạng của người dùng: Điện thoại/Laptop của người dùng bị rớt mạng Wi-Fi. Kết nối WebSocket bị đứt. Giao diện frontend hiện thông báo pop-up: "Đang cố gắng kết nối lại với máy chủ...". Khi có mạng, tự động Re-connect WebSocket và gọi API lấy trạng thái mới nhất để đồng bộ.

2.2.12 Use case 12: Phát cảnh báo sự cố

2.2.13 Use case 13: Dự báo lịch tưới tối ưu

2.3 Yêu cầu phi chức năng

UC1
Bảo mật: Mật khẩu tuyệt đối không được truyền dưới dạng plain-text qua mạng (phải dùng HTTPS) và không được lưu plain-text dưới Database.
Hiệu năng: Thời gian phản hồi từ lúc bấm nút đến lúc hiện thông báo thành công (không tính thời gian nhận email thực tế) phải < 2 giây.
UC2
Bảo mật: JWT bắt buộc phải được ký số (Signed) bằng thuật toán bảo mật mạnh (ví dụ: HS256 hoặc RS256) cùng một khóa bí mật (Secret Key) chỉ lưu trên Server để chống giả mạo token.
Hiệu năng: Quá trình xác thực và trả về Token phải được hoàn thành trong thời gian dưới 1 giây.
UC3
Tính khả dụng (Availability): Các thao tác liên quan đến cập nhật cấu trúc mạng (MQTT ACL) phải được thực hiện qua giao dịch (Transaction) để đảm bảo nếu lỗi mạng xảy ra ở giữa chừng, toàn bộ thao tác thêm mới sẽ được Rollback (hoàn tác).

3 Kiến trúc hệ thống

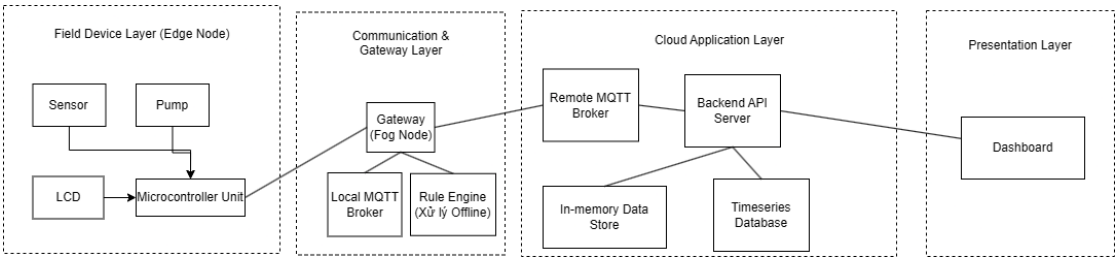


Figure 1: Sơ đồ khối hệ thống

Sơ đồ khối trên minh họa kiến trúc tổng thể của hệ thống Tưới tiêu thông minh dựa trên IoT (IoT-based Smart Irrigation), được xây dựng dựa trên mô hình chuẩn IoT 4 phân lớp. Luồng dữ liệu và tín hiệu điều khiển được luân chuyển chặt chẽ qua các lớp như sau:

1. **Lớp thiết bị thực (Field Device Layer / Edge Node):**
   Đây là phân lớp tương tác trực tiếp với môi trường vật lý bên trong nhà kính. Lõi xử lý tại đây là Vi điều khiển trung tâm. Vi điều khiển có nhiệm vụ liên tục đọc thông số từ các cảm biến như nhiệt độ, độ ẩm không khí, độ ẩm đất. Dựa trên dữ liệu thu thập được hoặc lệnh từ cấp trên, vi điều khiển sẽ xuất tín hiệu để đóng/ngắt các thiết bị như máy bơm, LED, đồng thời hiển thị trạng thái hệ thống tại chỗ thông qua màn hình (LCD).
2. **Lớp giao tiếp và trung chuyển (Communication & Gateway Layer):**
   Phân lớp này đóng vai trò là cầu nối mạng cục bộ (LAN) với Cloud Server. Fog Node nhận dữ liệu thô từ vi điều khiển thông qua giao tiếp MQTT. Điểm nhấn của kiến trúc này là sự xuất hiện của **Local MQTT Broker** hoạt động nội bộ. Nó giúp Fog Node và Vi điều khiển giao tiếp với nhau theo cơ chế *Pub/Sub* ngay tại nhà kính, đảm bảo hệ thống duy trì được khả năng vận hành tự động (*Offline mode*) ngay cả khi sự cố đứt cáp Internet xảy ra.
3. **Lớp ứng dụng điện toán đám mây (Cloud Application Layer):**
   Đây là bộ não xử lý dữ liệu quy mô lớn của toàn hệ thống. Dữ liệu từ Fog Node được luân chuyển lên Cloud qua Remote MQTT Broker thông qua môi trường Internet. Tại đây, máy chủ sẽ tiếp nhận các luồng dữ liệu để thực hiện các xử lý logic phức tạp, tính toán thời gian tưới, và ghi nhận toàn bộ lịch sử môi trường cũng như nhật ký thao tác vào Cơ sở dữ liệu. Máy chủ cũng làm nhiệm vụ cung cấp các API để giao tiếp với ứng dụng của người dùng.
4. **Lớp giao diện (Presentation Layer):**

   Đây là điểm tương tác trực tiếp với người dùng đầu cuối (Chủ nhà kính). Ứng dụng web cung cấp một bảng điều khiển trực quan hóa dữ liệu lịch sử dưới dạng biểu đồ, đồng thời cập nhật các thông số môi trường theo thời gian thực. Thông qua bảng điều khiển, người dùng có thể giám sát trạng thái hệ thống từ xa, thiết lập lại các ngưỡng, hoặc trực tiếp phát đi các lệnh điều khiển bật/tắt máy bơm thủ công. Các lệnh này sẽ được mã hóa và truyền ngược trở lại đi tới vi điều khiển.

3.1 Deployment Diagram

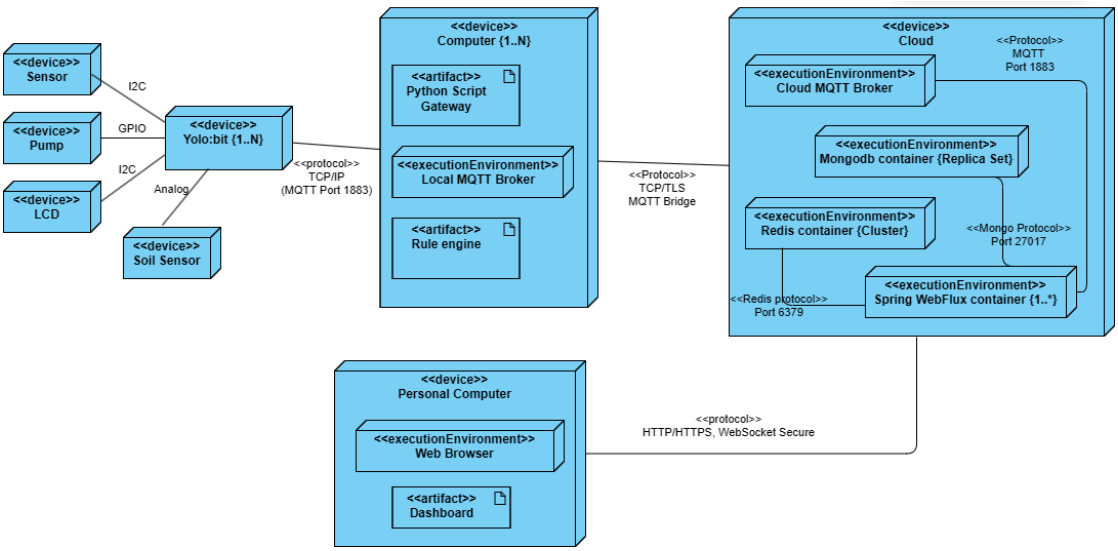


Figure 2: Sơ đồ triển khai hệ thống

Biểu đồ trên mô tả hệ thống được triển khai với hình thức vật lý.
Kiến trúc hệ thống IOT-Based Smart Irrigation in Greenhouse System được thiết kế theo mô hình Edge-Fog-Cloud. Hệ thống gồm 4 cụm thiết bị phân tán chính:

- Node Edge Device: Bao gồm vi điều khiển Yolo:bit, các cảm biến, màn hình LCD và máy bơm tại nhà kính.
- Node Fog Computer (Gateway): Máy tính nhúng/PC tại nhà kính, đóng vai trò xử lý logic và trung chuyển dữ liệu.
- Node Cloud Server: Nơi vận hành hệ thống dịch vụ backend, lưu trữ cơ sở dữ liệu và xử lý mô hình AI.
- Node Client Devices (Personal Computer): Thiết bị của người dùng (Chủ nhà kính/Quản trị viên) truy cập hệ thống qua trình duyệt web.

Hệ thống trên có thể dễ dàng mở rộng theo chiều rộng như:

-

3.1.1 Node Edge Device

Vai trò: Là thiết bị vi điều khiển thu thập dữ liệu và trực tiếp điều khiển bơm tại không gian nhà kính.
Các thành phần và giao tiếp:

- Giao tiếp với Sensor (DHT20) và LCD thông qua chuẩn giao tiếp nối tiếp đồng bộ I2C.
- Giao tiếp với Soil Sensor thông qua tín hiệu tương tự Analog.
- Truyền tín hiệu điều khiển tới Pump thông qua GPIO.

Kết nối mạng: Kết nối với Node Fog Computer (Gateway) thông qua mạng Wi-Fi cục bộ (LAN) bằng giao thức TCP/IP (MQTT Port 1883).

3.1.2 Node Fog Computer (Gateway)

Vai trò: Đóng vai trò là thiết bị trung gian xử lý dữ liệu, trung chuyển dữ liệu, xử lý logic và đảm bảo hệ thống vận hành an toàn khi mất kết nối Internet.
Môi trường thực thi chứa một Local MQTT Broker để làm trung tâm nội bộ cho các thiết bị Yolo:bit.
Artifacts:

- Python Script Gateway: Thu thập, chuyển đổi định dạng bản tin thành chuẩn (SenML) và quản lý kết nối lên Cloud. (Có thể thay bằng NODE-RED)
- Rule Engine: Bộ xử lý quy tắc cục bộ, đánh giá dữ liệu cảm biến để ra quyết định bật/tắt bơm tự động dựa trên cấu hình đã lưu. (Có thể thay bằng NODE-RED)

Kết nối mạng: Giao tiếp với Cloud Broker thông qua cơ chế MQTT Bridge (hoặc Hub and Spoke) sử dụng đường truyền bảo mật TCP/TLS.

3.1.3 Node Cloud Server

Vai trò: Xử lý nghiệp vụ cốt lõi, lưu trữ dữ liệu lớn và cung cấp API cho ứng dụng người dùng. Hệ thống được triển khai dưới dạng các Container độc lập để dễ dàng mở rộng.
Thành phần:

- Cloud MQTT Broker: Điểm tiếp nhận dữ liệu toàn cầu từ các Gateway.
- Spring WebFlux container: Máy chủ Backend xử lý luồng dữ liệu bất đồng bộ (Non-blocking).
- Redis container: Đóng vai trò In-memory Cache xử lý luồng dữ liệu thời gian thực (Hot Path) qua giao thức nội bộ (Port 6379).
- Mongodb container: Cơ sở dữ liệu chuỗi thời gian (Time-series) lưu trữ dữ liệu lịch sử và nhật ký (Cold Path) qua Mongo Protocol (Port 27017).

3.1.4 Node Client Devices (Personal Computer)

Vai trò: Điểm chạm cuối cùng, cung cấp giao diện trực quan cho Chủ nhà kính và Quản trị viên.
Môi trường thực thi: Hoạt động trên nền tảng Web Browser.
Artifacts: Ứng dụng Dashboard (được xây dựng bằng React).
Kết nối mạng: Giao tiếp với Cloud Backend thông qua các giao thức bảo mật: HTTP/HTTPS cho các yêu cầu REST API thông thường, và WebSocket Secure (WSS) để nhận luồng dữ liệu cập nhật theo thời gian thực.

3.2 Luồng dữ liệu

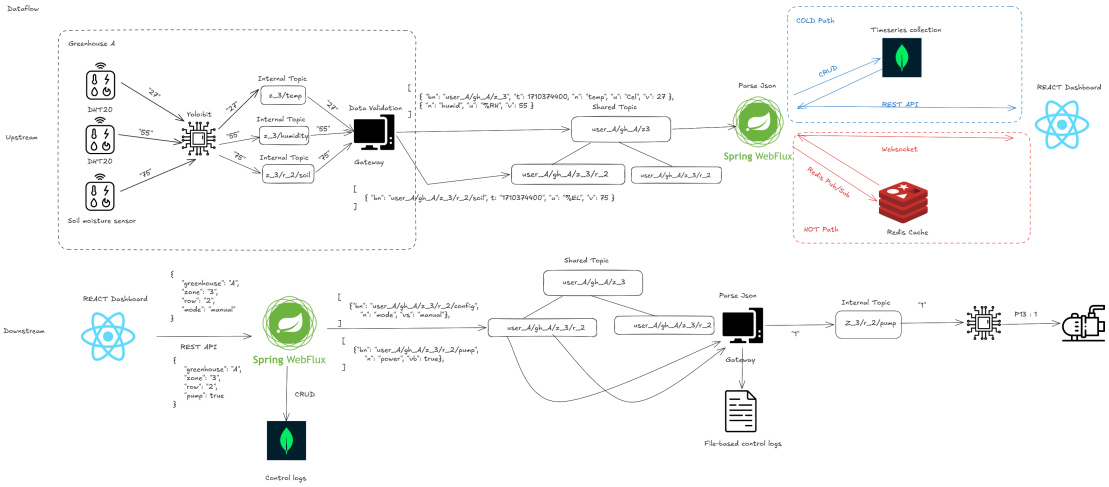


Figure 3: Sơ đồ luồng dữ liệu của hệ thống

Sơ đồ trên mô tả luồng dữ liệu của hệ thống theo 2 tình huống: từ các thiết bị biên đẩy lên hệ thống trung tâm (upstream), các lệnh điều khiển từ hệ thống trung tâm đẩy xuống thiết bị biên (downstream)

3.2.1 Luồng dữ liệu thu thập (upstream)

Đây là hành trình dữ liệu môi trường đi từ nhà kính lên bảng điều khiển của người dùng:

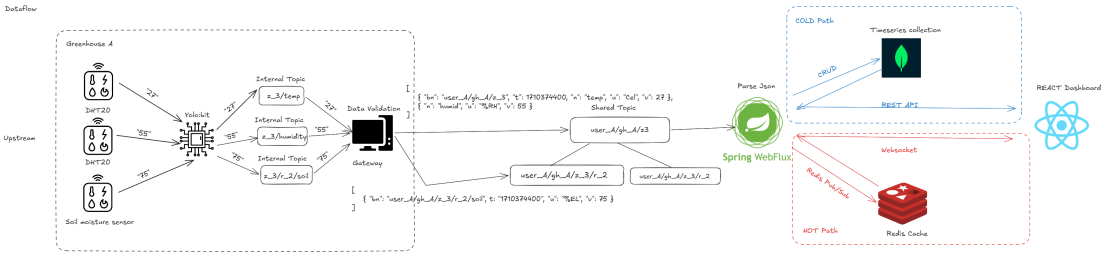


Figure 4: Sơ đồ luồng dữ liệu upstream của hệ thống

Bước 1: Thu thập dữ liệu thô (Raw Data Collection)
Các cảm biến môi trường (*DHT20* đo nhiệt độ/độ ẩm không khí, độ ẩm đất) liên tục đọc các thông số vật lý và truyền tín hiệu về vi điều khiển **Yolo:bit**.

Bước 2: Phân phối mạng nội bộ (Internal MQTT Topic)
Yolo:bit đẩy các giá trị thô này lên *Local MQTT Broker* (đặt tại nhà kính) thông qua các `Internal Topic` tương ứng như `z_3/temp`, `z_3/humidity`, và `z_3/r_2/soil`.

Bước 3: Xử lý tại Fog (Fog Processing & Validation)
Máy tính **Gateway** lắng nghe các `Internal Topic` này. Nó thực hiện xác thực dữ liệu (*Data Validation*), sau đó đóng gói các giá trị thô thành chuẩn **JSON (SenML format)** bao gồm: tên định danh (`bn`), loại dữ liệu (`n`), đơn vị (`u`), giá trị (`v`) và mốc thời gian (`t`) ...

Bước 4: Đồng bộ lên Đám mây (Cloud Sync)
Gateway đẩy (*Publish*) các mảng JSON này qua môi trường Internet lên các `Shared MQTT Topic` (`user_A/gh_A/z3`, `user_A/gh_A/z_3/r_2`), ... trên *Cloud MQTT Broker*.

Bước 5: Xử lý tại Backend (Spring WebFlux)
Máy chủ **Spring WebFlux** nhận và phân tích (*Parse*) chuỗi JSON. Tại đây, luồng dữ liệu được chia làm hai nhánh:

- **COLD Path:** Dữ liệu được lưu trữ vào **MongoDB** (*Timeseries collection*) phục vụ truy xuất lịch sử, vẽ biểu đồ hoặc huấn luyện AI.
- **HOT Path:** Dữ liệu được đẩy vào **Redis Cache**. Hệ thống **WebSockets** sẽ đẩy trực tiếp dữ liệu từ Redis lên giao diện *React Dashboard* theo thời gian thực (*Real-time*).

3.2.2 Luồng lệnh điều khiển (downstream)

Đây là quy trình xử lý lệnh điều khiển từ người dùng tới ứng dụng để tác động lên phần cứng:

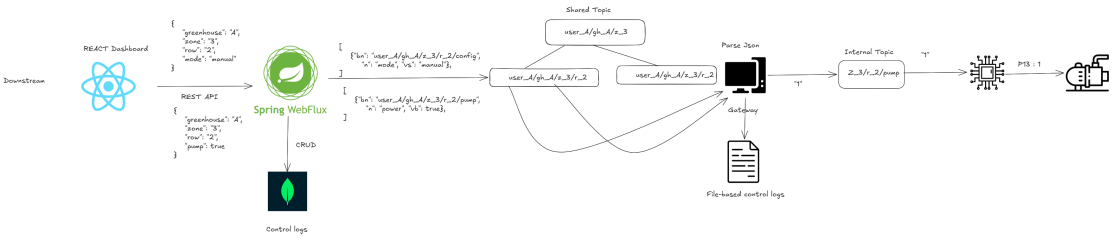


Figure 5: Sơ đồ luồng dữ liệu downstream của hệ thống

Bước 1: Người dùng ra lệnh (User Command)
Người dùng thao tác trên Dashboard (ví dụ: bấm nút bật máy bơm). Giao diện tạo một **HTTP REST API** mang *payload* JSON cấu hình gửi lên máy chủ Backend.

Bước 2: Xử lý và lưu trữ tại Backend
Spring WebFlux tiếp nhận request và thực hiện song song:

- Lưu hành động vào **MongoDB** (*Control logs*) để phục vụ truy xuất.
- Biên dịch lệnh thành mảng JSON chuẩn **SenML** (Ví dụ: `{"bn": "user_A/gh_A/z_3/r_2/pum "n": "power", "vb": true}`).

Bước 3: Truyền lệnh xuống Fog Node (Cloud to Fog)
Máy chủ *Publish* mảng JSON này vào *Cloud MQTT Broker* tại topic `user_A/gh_A/z_3/r_2`.

Bước 4: Phiên dịch tại Fog Node
Gateway nhận bản tin, phân tích lệnh, ghi nhật ký nội bộ dự phòng, sau đó "dịch" lệnh JSON phức tạp này thành tín hiệu điều khiển đơn giản (ví dụ: chuỗi `"1"`).

Bước 5: Thực thi phần cứng (Hardware Execution)
Gateway đẩy tín hiệu `"1"` vào `Internal Topic` (`z_3/r_2/pump`). Vi điều khiển **Yolo:bit** nhận lệnh, lập tức xuất tín hiệu mức cao (*High*) ra chân vật lý **P13**, kích hoạt **Relay** để bật máy bơm.

3.3 ERD

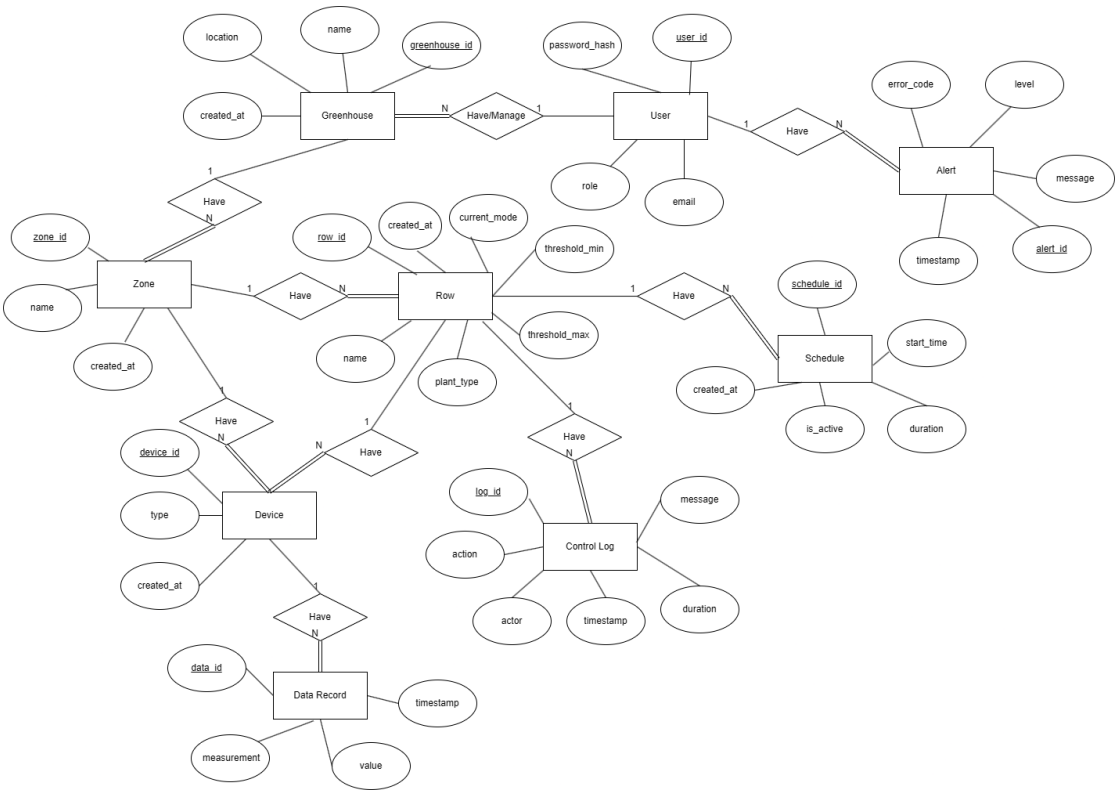


Figure 6: Sơ đồ ERD của cơ sở dữ liệu

Sơ đồ trên mô tả mối quan hệ giữa các thực thể trong hệ thống theo 4 nhóm như sau:

3.3.1 Nhóm quản lý tài khoản và không gian

Nhóm này đảm nhiệm việc phân lập dữ liệu và quản lý cấu trúc không gian vật lý của nhà kính.

- **Thực thể User (Người dùng):** Đại diện cho chủ sở hữu nhà kính hoặc quản trị viên. Các thuộc tính bao gồm `user_id` (*Khóa chính*), `email`, `password_hash` (mật khẩu đã mã hóa) và `role` (phân quyền).
- **Thực thể Greenhouse (Nhà kính):** Quản lý thông tin tổng quan của các nhà kính vật lý. Có **mối quan hệ 1-N với User** (Một người dùng quản lý nhiều nhà kính). Bao gồm các trường `greenhouse_id` (*Khóa chính*), `name`, `location`, `created_at`.
- **Thực thể Zone (Phân khu):** Chia nhỏ nhà kính thành các khu vực quản lý vi khí hậu chung. Có **mối quan hệ 1-N với Greenhouse**.
- **Thực thể Row (Hàng cây):** Đơn vị quản lý tưới tiêu nhỏ nhất. Trực thuộc Zone (**Quan hệ 1-N**). Chứa cấu hình logic: `current_mode` (Auto/Manual/AI/Schedule), `threshold_min`, `threshold_max` và `plant_type`.

3.3.2 Nhóm quản lý thiết bị và đo lường

- **Thực thể Device (Thiết bị Biên):** Quản lý định danh phần cứng (Cảm biến, Máy bơm). Device nhận **mối quan hệ 1-N** từ cả Zone (cho thiết bị chung) và Row (cho thiết bị riêng lẻ).
- **Thực thể Data Record (Dữ liệu đo lường):** Lưu trữ dữ liệu chuỗi thời gian (*Time-series*). Có **mối quan hệ 1-N với Device**. Mỗi bản ghi chứa `data_id`, `measurement` (loại thông số), `value` và `timestamp`.

3.3.3 Nhóm điều khiển và tự động hóa

- **Thực thể Schedule (Lịch trình):** Quản lý cấu hình hẹn giờ tưới cho từng hàng cây. Có **quan hệ 1-N với Row**. Chứa các thuộc tính `start_time`, `duration` và `is_active`.
- **Thực thể Control Log (Nhật ký điều khiển):** Đóng vai trò kiểm toán hệ thống, lưu lại mọi hành động tác động lên phần cứng của Row (**Quan hệ 1-N**). Bao gồm `action` (ON/OFF), `actor` (User/AI/Schedule), `duration` và `timestamp`.

3.3.4 Nhóm giám sát an toàn

- **Thực thể Alert (Cảnh báo):** Ghi nhận sự cố kỹ thuật khẩn cấp (Cạn nước, lỗi cảm biến, mất kết nối). Có **mối quan hệ 1-N trực tiếp với User** để đảm bảo thông báo đến đúng chủ sở hữu. Bao gồm `alert_id` (*Khóa chính*), `level` (mức độ), `error_code`, `message` và `timestamp`.

3.4 Cấu trúc MQTT Topic Tree

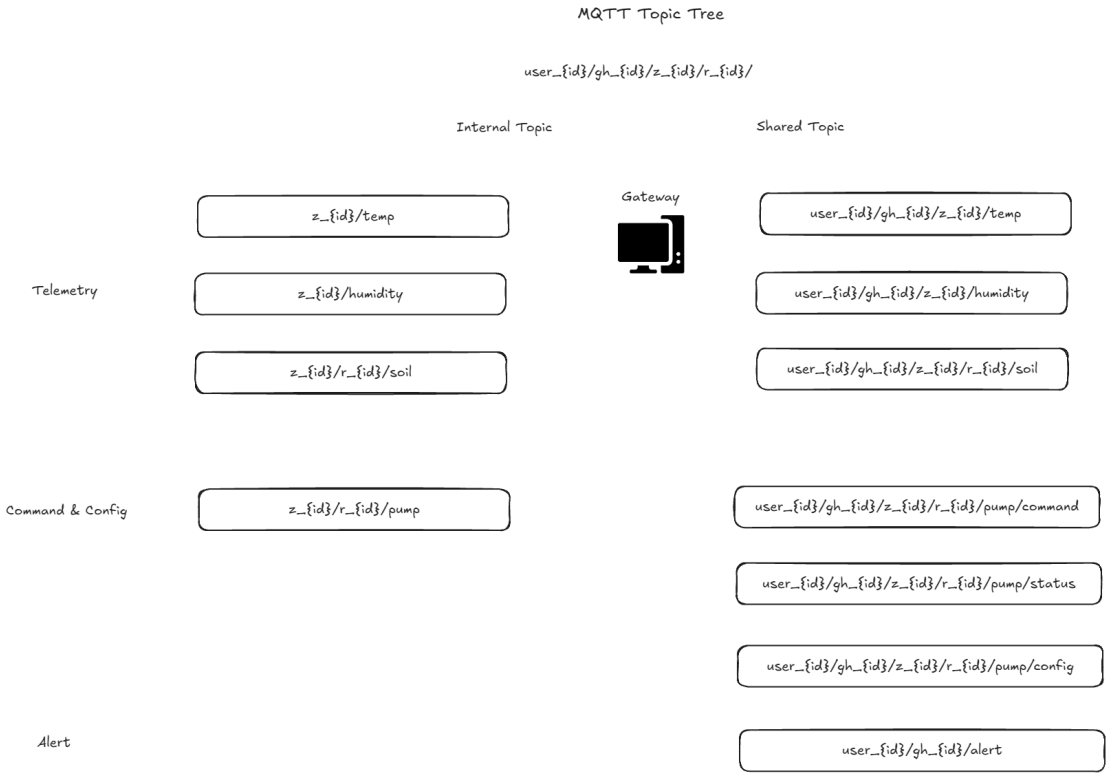


Figure 7: Các topic của mqtt broker của hệ thống

Sơ đồ MQTT Topic Tree minh họa cấu trúc topic của hệ thống. Cấu trúc này áp dụng mô hình Hierarchical Namespace dựa trên định dạng chuẩn: user_{user_id}/gh_{greenhouse_id}/z_{zone_id} nhằm đảm bảo khả năng định danh chính xác đến từng thiết bị vật lý trong môi trường đa khách hàng.
Điểm cốt lõi của kiến trúc này là sự phân lập mạng được thực hiện bởi thiết bị Gateway, chia hệ thống thành hai nửa:

- Internal Topic (Bên trái): Không gian giao tiếp nội bộ tại nhà kính (LAN). Các topic được thiết kế cực kỳ ngắn gọn (chỉ bắt đầu từ z_{zone_id}) nhằm tối ưu hóa bộ nhớ RAM và băng thông hạn hẹp của vi điều khiển (Yolo:bit).
- Shared Topic (Bên phải): Không gian giao tiếp trên Đám mây (Cloud). Các topic có đầy đủ tiền tố định danh (user_{user_id}/gh_{greenhouse_id}/...) để Cloud Broker phân luồng dữ liệu an toàn giữa các chủ nhà kính khác nhau.

3.4.1 Chi tiết phân luồng các nhóm chủ đề

- **Nhóm dữ liệu đo lường:**
  Thiết bị biên liên tục gửi thông số môi trường qua các Internal Topic.

– Dữ liệu cấp khu vực (Zone): Gồm nhiệt độ và độ ẩm được gửi qua z_{zone_id}/temp và z_{zone_id}/humidity.
– Dữ liệu cấp hàng (Row): Gồm độ ẩm đất được gửi qua z_{zone_id}/r_{row_id}/soil.
Vai trò Gateway: Đóng vai trò là cầu nối, tự động gắn thêm tiền tố user_{user_id}/gh_{greenhouse_id}/ vào các bản tin này, định dạng lại payload theo chuẩn SenML và đẩy lên Shared Topic tương ứng trên Cloud.

- **Nhóm lệnh và cấu hình:**
Đây là nhóm thể hiện rõ nhất sức mạnh xử lý của Gateway trong việc giảm tải cho thiết bị đầu cuối. Trên Cloud, hệ thống phân chia rất chi tiết thành 3 luồng riêng biệt:
  - Lệnh điều khiển (.../pump/command)
  - Trạng thái phản hồi (.../pump/status)
  - Cấu hình vận hành (.../pump/config)

  Vai trò Gateway: Thay vì bắt mạch Yolo:bit phải lắng nghe (Subscribe) cả 3 topic phức tạp trên, Gateway sẽ tiếp nhận toàn bộ, xử lý logic (Parse JSON), và gom chúng lại thành các tín hiệu đơn giản truyền qua một Internal Topic duy nhất là z_{zone_id}/r_{row_id}/pump. Ngược lại, khi Yolo:bit phản hồi trạng thái, Gateway sẽ tự động phân loại và publish ngược lên topic status trên Cloud.

- **Nhóm cảnh báo (Alert):**
Dành riêng cho luồng sự kiện khẩn cấp như sự cố phần cứng, cạn bể nước hoặc lỗi cảm biến.
Vai trò Gateway: Gateway tự động giám sát trạng thái của mạng nội bộ và sức khỏe thiết bị. Khi phát hiện bất thường, nó trực tiếp tổng hợp và gửi bản tin báo cáo lên Cloud thông qua topic user_{user_id}/gh_{greenhouse_id}/alert để kích hoạt thông báo tức thời cho người dùng qua ứng dụng.

3.5 Sequence diagram

3.6 Activity diagram

4 Đánh giá và kiểm thử hệ thống

Để đảm bảo tính khả thi và độ tin cậy của giải pháp, quá trình kiểm thử được tiến hành song song trên hai phương diện: độ ổn định của phần mềm/phần cứng hệ thống và hiệu quả thực tế mang lại cho cây trồng.

4.1 Kiểm thử hệ thống

Phần này tập trung đánh giá chất lượng mã nguồn và các chỉ số hiệu năng của kiến trúc Edge-Fog-Cloud.

4.1.1 Kiểm thử phần mềm theo các phương pháp thiết kế Test Case

Để đảm bảo phần mềm hoạt động ổn định, bộ Test Case được xây dựng dựa trên các kỹ thuật kiểm thử kinh điển trong Kỹ nghệ Phần mềm, áp dụng trực tiếp vào các Use-case của hệ thống:

1. **Phân vùng tương đương (Equivalence Partitioning):**
   - *Mục tiêu:* Kiểm tra logic cấu hình ngưỡng (UC07).
   - *Test case:* Chia dải độ ẩm 0–100% thành 3 vùng: Vùng khô ($< 50\%$), Vùng an toàn (50%–80%), Vùng úng ($> 80\%$). Nhập các giá trị đại diện (30, 65, 90) để xác nhận Edge Yolo:bit đóng/ngắt bơm đúng logic.

2. **Phân tích giá trị biên (Boundary Value Analysis):**
   - *Mục tiêu:* Kiểm tra giới hạn kiểu dữ liệu.
   - *Test case:* Nhập cấu hình `threshold_min = 0` và `threshold_max = 100` hoặc test lịch tưới với `duration = 0` và `1440` (24h) để kiểm tra khả năng xử lý tại biên.

3. **Kiểm thử chuyển trạng thái (State Transition Testing):**
   - *Mục tiêu:* Kiểm tra sự trơn tru khi thay đổi chế độ vận hành (UC07, UC08).
   - *Test case:* Chuyển đổi liên tục giữa AI Mode → MANUAL ON → MANUAL OFF → AUTO để kiểm tra Relay có bị "treo" hay không.

4. **Kiểm thử tiêu cực (Negative Testing):**
   - *Mục tiêu:* Đánh giá khả năng chặn dữ liệu lỗi của REST API.
   - *Test case:* Truyền payload JSON sai định dạng (ví dụ: `duration = "mười phút"`) hoặc cấu hình `min > max` để kiểm tra phản hồi `HTTP 400 Bad Request`.

5. **Kiểm thử xử lý Ngoại lệ / Chịu lỗi (Failover Testing):**
   - *Mục tiêu:* Kiểm tra chế độ "sinh tồn" của thiết bị Biên (UC09).
   - *Test case:* Ngắt kết nối Wi-Fi khi đang tưới AUTO. Theo dõi Yolo:bit có tự động duy trì vòng lặp tưới cục bộ và ngắt bơm khi đạt ngưỡng an toàn hay không.

6. **Kiểm thử mức Đơn vị (Unit Testing):** Sử dụng thư viện *JUnit/Mockito* để kiểm thử độc lập hàm `calculateIrrigationTime()` của module AI với các bộ dữ liệu mẫu mà không cần kết nối cơ sở dữ liệu.

7. **Kiểm thử Tích hợp (Integration Testing):** Đo đạc thời gian từ khi người dùng nhấn "Bật bơm" trên Dashboard đến khi Yolo:bit thực thi đóng Relay qua luồng: *React* $\rightarrow$ *Spring WebFlux* $\rightarrow$ *MQTT* $\rightarrow$ *Gateway* $\rightarrow$ *Edge*.

8. **Kiểm thử Bảo mật (Security/Auth Testing):** Sử dụng *JWT Token* của tài khoản A để truy cập dữ liệu của nhà kính thuộc tài khoản B. Hệ thống bắt buộc phải trả về lỗi `HTTP 403 Forbidden`.

9. **Kiểm thử Khóa phần cứng (Hardware Interlock Testing):** Giả lập sự cố cạn bể nước (nhấc cảm biến mực nước). Khi đó, mọi lệnh `MANUAL ON` từ Dashboard phải bị từ chối và hiển thị cảnh báo an toàn.

4.1.2 Đánh giá Hiệu năng Hệ thống (System Performance)

- **Kiểm thử Độ trễ (Latency Test):** Do đạc luồng *Hot Path* (Redis + WebSockets). Mục tiêu: Thời gian từ cảm biến đến Dashboard phải < 500ms.
- **Kiểm thử Tải (Load/Stress Testing):** Sử dụng *Apache JMeter* giả lập 1.000 thiết bị Edge gửi dữ liệu liên tục (1 bản tin/giây) để xác định ngưỡng nghẽn (*Bottleneck*) của Server.
- **Kiểm thử Khôi phục (Recovery Test):** Mô phỏng sập nguồn Gateway và đo thời gian tái kết nối MQTT cũng như đồng bộ cấu hình từ Cloud.

4.2 Đánh giá độ hiệu quả cây trồng

Các thực nghiệm nông nghiệp được tiến hành để chứng minh tính hiệu quả của thuật toán tưới và sự phát triển thực tế của cây trồng.

4.2.1 Phương pháp "Cái cân"

Đây là phương pháp tiêu chuẩn dùng để hiệu chuẩn độ chính xác của cảm biến và thuật toán tưới. Lượng nước cây tiêu thụ được xác định qua công thức:

$$W_{consumed} = W_{t-1} - W_t \tag{1}$$

Trong đó W_{t-1} là trọng lượng chậu cây ngày hôm trước và W_t là trọng lượng ngày hiện tại. Hệ thống được đánh giá là hiệu quả nếu trọng lượng chậu luôn duy trì ổn định ở mức $60\% - 80\%$ giữa trọng lượng đất khô (W_{dry}).

4.2.2 Phân tích đồ thị "Răng cưa" (Soil Moisture Profile)

Dữ liệu độ ẩm đất trên Dashboard được phân tích dựa trên 3 tiêu chí:

- **Độ dốc tiêu hao (Âm):** Thể hiện sự hô hấp và tiêu thụ nước tự nhiên của cây vào ban ngày.

- **Phản ứng tức thì (Dương):** Thể hiện độ nhạy của cảm biến và vị trí vòi tưới khi máy bơm kích hoạt.
- **Độ chuẩn xác của Ngưỡng:** Đảm bảo hệ thống kích hoạt tưới ngay tại điểm *Threshold Min*, tránh tình trạng cây bị héo do thiếu nước.

4.2.3 Thiết lập mô hình đối chứng (A/B Testing Thực địa)

Khu vực thực nghiệm được chia làm hai lô độc lập với cùng giống cây và điều kiện ánh sáng:

- **Lô A (Đối chứng):** Chăm sóc theo phương pháp truyền thống (tưới thủ công hoặc hẹn giờ cố định).
- **Lô B (Thử nghiệm):** Chạy hoàn toàn bằng hệ thống *Smart Irrigation* (tưới theo độ ẩm thực tế và hệ số K_c).

Kết quả được đánh giá dựa trên tổng lượng nước tiêu thụ và năng suất sinh khối sau một chu kỳ thu hoạch.

4.2.4 Đánh giá bằng Dữ liệu giả lập (Backtesting cho AI)

Thuật toán dự báo lịch tưới (ET_c) được kiểm thử trên môi trường giả lập trước khi triển khai:

a) Sử dụng dữ liệu vi khí hậu lịch sử từ *OpenWeatherMap API*.
b) Truy xuất bốc thoát hơi nước tham chiếu (ET_o) qua phương trình *Penman-Monteith* (thư viện `pyet`).
c) So sánh kết quả của AI với phần mềm tiêu chuẩn **CROPWAT 8.0** của FAO. Thuật toán đạt yêu cầu khi sai số nhỏ hơn 5%.

4.3 Kết quả

TÀI LIỆU THAM KHẢO

[1] Steve Cope. Mosquitto bridge configuration - working example. http://www.steves-internet-guide.com/mosquitto-bridge-configuration/, 2021. Accessed: 2025-03-14.

[2] Food and Agriculture Organization of the United Nations. The state of food and agriculture 2022. leveraging automation in agriculture for transforming agrifood systems. Technical report, FAO, Rome, Italy, 2022.

[3] UNESCO World Water Assessment Programme. The united nations world water development report 2024: Water for prosperity and peace. Technical report, UNESCO, Paris, France, 2024. Accessed: 2025-03-12.